create a tech stack doc for the project

Here is a comprehensive Tech Stack Document for the Rentro project, aligning with the architectural and design considerations previously discussed.

Tech Stack Document: Rentro - PG Accommodation Platform

Document Version: 1.0
Date: March 28, 2026
Prepared By: [Your Name/Team Lead]
Approved By: [CTO/Project Sponsor]

---

1. Introduction

This document details the complete technology stack for the Rentro web application. It outlines the specific programming languages, frameworks, libraries, databases, cloud services, and development tools selected to build a professional, scalable, secure, and high-performance platform for PG accommodations. The choices are driven by the project's functional and non-functional requirements, emphasizing a modern, maintainable, and efficient development ecosystem.

2. Guiding Principles for Technology Selection

The following principles guided the selection of our technology stack:

- **Scalability:** Technologies capable of handling growth in users, data, and traffic.
- **Performance:** Fast response times and efficient resource utilization.
- **Reliability & High Availability:** Robust solutions with strong community support and proven stability.
- **Security:** Features and practices that facilitate building a secure application.
- **Maintainability & Developer Experience:** Clean code, strong ecosystem, good documentation, and active community for easier development and long-term support.

🚀 Powered by Yupp

- **Extensibility:** Ease of integrating new features and third-party services.
- **Cost-Effectiveness:** Optimize for operational costs while meeting performance and reliability targets.
- **Cloud-Native:** Leverage managed cloud services to reduce operational overhead.

3. Overall Architecture

Rentro will be built as a **microservices-oriented application** deployed on a **cloud-native platform**. The frontend will be a **Single-Page Application (SPA)** framework with Server-Side Rendering (SSR) capabilities, consuming **RESTful APIs** exposed by the backend services.

4. Technology Stack Breakdown

4.1. Frontend (Client-side)

- **Framework:**
  - **Next.js (React Framework):** Chosen for its strong React ecosystem, built-in Server-Side Rendering (SSR) for improved SEO and initial page load performance, static site generation (SSG), and API routes capabilities.
- **Language:**
  - **TypeScript:** Provides static typing, enhancing code quality, maintainability, and developer experience by catching errors early.
- **Styling:**
  - **Tailwind CSS:** A utility-first CSS framework that enables rapid UI development and ensures design consistency, aligning well with the Dribbble design reference.
  - **PostCSS:** For processing Tailwind CSS and other CSS transformations.
- **State Management:**
  - **React Context API / Zustand:** For local and global state management within the React components. Zustand is a lightweight alternative to Redux, offering simplicity and performance.
- **Form Management:**
  - **React Hook Form / Zod:** Efficient form validation and management. Zod for schema validation.
- **HTTP Client:**
  - **Axios / Fetch API:** For making API requests from the frontend.
- **Linting & Formatting:**

🚀 Powered by Yupp

- **ESLint, Prettier:** Maintain code quality and consistent formatting.

4.2. Backend (Server-side & API)

- **Framework:**
  - **NestJS:** A progressive Node.js framework for building efficient, reliable, and scalable server-side applications. It uses TypeScript, supports modular architecture (ideal for microservices), and provides excellent structure for enterprise-grade applications.
- **Language:**
  - **TypeScript:** Consistent with the frontend, enabling type safety across the full stack.
- **Runtime Environment:**
  - **Node.js:** JavaScript runtime for executing backend code.
- **Database ORM/ODM:**
  - **TypeORM / Prisma:** For object-relational mapping, simplifying database interactions and enabling type-safe queries with PostgreSQL.
- **Validation:**
  - **Class-validator:** Integrated with NestJS for declarative request payload validation.
- **API Documentation:**
  - **Swagger/OpenAPI:** For generating interactive API documentation, aiding frontend-backend communication and external integrations.

4.3. Database & Data Management

- **Primary Database (Relational):**
  - **PostgreSQL:** A powerful, open-source relational database known for its robustness, ACID compliance, extensibility, and advanced features (e.g., JSONB support for flexible schemas, strong indexing capabilities).
- **Caching & Session Management:**
  - **Redis:** An in-memory data structure store used as a cache, message broker (Pub/Sub for internal events), and for managing user sessions (e.g., JWT token blacklisting, refresh token storage).
- **Search Engine:**
  - **Elasticsearch:** A highly scalable full-text search and analytics engine. Essential for providing fast, relevant, and complex search queries across PG listings.
- **Object Storage (Media Files):**

🚀 Powered by Yupp

- **AWS S3 (or equivalent GCP Cloud Storage):** For storing large binary objects such as PG images, videos, and user profile pictures. Offers high availability, durability, and scalability.

4.4. Cloud Infrastructure & Platform

- **Cloud Provider:**
  - **Amazon Web Services (AWS):** Offers a comprehensive suite of services, extensive documentation, and a mature ecosystem for highly scalable and reliable applications. (Alternatively, Google Cloud Platform (GCP) or Microsoft Azure could be considered based on team expertise and specific feature needs).
- **Compute:**
  - **AWS Fargate (for ECS):** Serverless compute for containers, abstracting away server management for backend services and Next.js applications. Provides easy scaling and reduced operational overhead.
- **Container Orchestration:**
  - **AWS Elastic Container Service (ECS):** A fully managed container orchestration service, simpler to manage than Kubernetes for initial microservices deployment, particularly with Fargate.
- **Load Balancing:**
  - **AWS Application Load Balancer (ALB):** Distributes incoming traffic across multiple targets, providing high availability and fault tolerance.
- **API Gateway:**
  - **AWS API Gateway:** Acts as a single entry point for all API requests, providing traffic management, authorization, access control, and API version management.
- **Managed Databases:**
  - **AWS RDS (PostgreSQL):** Fully managed relational database service.
  - **AWS ElastiCache (Redis):** Fully managed Redis service.
  - **AWS OpenSearch Service (Elasticsearch):** Fully managed Elasticsearch service.
- **Content Delivery Network (CDN):**
  - **AWS CloudFront:** For caching and delivering static assets (from S3) closer to users, reducing latency and improving load times.
- **DNS Management:**
  - **AWS Route 53:** Scalable cloud Domain Name System (DNS) web service.

🚀 Powered by Yupp

4.5. DevOps & Tooling

- **Version Control:**
  - **Git (hosted on GitHub):** Distributed version control system for source code management and collaboration.
- **Continuous Integration/Continuous Deployment (CI/CD):**
  - **GitHub Actions:** Automate build, test, and deployment workflows directly from GitHub repositories.
- **Containerization:**
  - **Docker:** For packaging applications and their dependencies into portable containers, ensuring consistent environments.
- **Infrastructure as Code (IaC):**
  - **Terraform:** For defining and provisioning cloud infrastructure in a declarative way, ensuring consistency and repeatability.
- **Monitoring & Logging:**
  - **AWS CloudWatch:** For collecting and tracking metrics, collecting and monitoring log files, and setting alarms.
  - **Prometheus & Grafana (Optional for MVP, good for scaling):** For advanced metrics collection, time-series data storage, and custom dashboard visualization.
  - **Sentry:** For real-time error tracking and alerting in both frontend and backend.
- **Distributed Tracing (Optional for MVP, good for debugging microservices):**
  - **AWS X-Ray / Jaeger:** For end-to-end tracing of requests across microservices.

4.6. Third-Party Integrations

- **Email Service:**
  - **AWS Simple Email Service (SES) / SendGrid:** For transactional emails (e.g., account verification, password resets, inquiry notifications).
- **Mapping Service:**
  - **Google Maps Platform APIs:** For geo-coding addresses, displaying maps on listing pages, and calculating distances/proximity.
- **Payment Gateway (Phase 2):**
  - **Stripe / Razorpay / PayU:** For secure online payment processing, tokenization, and recurring billing. Selection will depend on geographic targets and specific payment

🚀 Powered by Yupp

needs.

4.7. Development Tools & Utilities

- **IDE:**
  - **Visual Studio Code (VS Code):** Popular, extensible IDE with excellent TypeScript support.
- **Package Manager:**
  - **npm / Yarn:** For managing project dependencies.
- **Testing Frameworks:**
  - **Jest (Unit/Integration):** For JavaScript/TypeScript testing.
  - **React Testing Library (Frontend UI):** For testing React components.
  - **Cypress / Playwright (E2E):** For end-to-end testing of user flows.

5. Justification and Rationale

- **Next.js & React:** Provides a modern, component-based UI, excellent developer tooling, strong community support, and crucial SSR capabilities for SEO and performance for a marketplace.
- **NestJS & Node.js/TypeScript:** Offers a robust, scalable, and structured approach for microservices development. TypeScript ensures code quality and maintainability across the full stack. Node.js's event-driven, non-blocking I/O model is well-suited for high-throughput APIs.
- **PostgreSQL:** Chosen for its reliability, data integrity (ACID), advanced features, and strong support for various data types, making it a solid choice for a core business database.
- **Elasticsearch:** Essential for a property listing platform where fast and flexible text-based search is a core feature.
- **AWS Ecosystem:** Provides a comprehensive, mature, and highly scalable set of managed

🚀 Powered by Yupp